

Implementation Plan: Caching & Optimization Suite for Colab T4

Goal Description

Optimize the DiffNorm-Contact HMR training pipeline for Google Colab Tesla T4 hardware. Address runtime bottlenecks that caused training to take ~8.9s per batch (dominated by uncached 1920×1080 JPEG decoding and unvectorized screen-space tile loops), reduce epoch runtime from ~3.5 hours down to ~8 minutes (>25x speedup), and implement resilient cross-session stateful checkpointing to seamlessly navigate Colab free tier's 5-hour session allocation limit.

Model Architecture & Size Analysis

1. Parameter Footprint & Memory Analysis

The DiffNorm-Contact HMR model couples a canonical SMPL kinematic chain with 6,890 vertex-anchored tangential 3D Gaussian disks. Because it operates through an explicit geometric representation rather than a multi-billion-parameter neural network, its parameter footprint is **exceptionally compact**:

Component	Tensor Dimensions	Precision	Parameter Count	Memory on GPU
SMPL Kinematics (θ)	(24, 3) axis-angle	float32	72	288 Bytes
Global Translation ($\mathbf{t}$)	(3,) XYZ	float32	3	12 Bytes
Gaussian Offsets ($\delta\mu$)	(6890, 3) local offsets	float32	20,670	82.68 KB
Tangent Scales ($\mathbf{s}$)	(6890, 2) log s_1, s_2	float32	13,780	55.12 KB
Orientations ($\mathbf{q}$)	(6890, 4) unit quaternions	float32	27,560	110.24 KB
Opacities (α)	(6890, 1) raw logits	float32	6,890	27.56 KB
Colors ($\mathbf{c}$)	(6890, 3) RGB diffuse	float32	20,670	82.68 KB
TOTAL MODEL PARAMETERS	—	—	89,645	~350.2 KB

2. Complete Training State Footprint

When checkpointing the model during training, the checkpoint stores: 1. **Model parameters** ($\theta, \mathbf{t}, \delta\mu, \mathbf{s}, \mathbf{q}, \alpha, \mathbf{c}$): ~350 KB 2. **AdamW Optimizer States** (first moment $\mathbf{m}$ and second moment $\mathbf{v}$ for kinematic and deformation parameters): $2 \times 350 \text{ KB} \approx 700 \text{ KB}$ 3. **Training Metadata** (`epoch` , `batch_idx` , `best_loss` , scheduler state): ~2 KB - **Total Checkpoint File Size** (`.pt`): $\approx 1.15 \text{ MB}$ uncompressed ($\approx 400 \text{ KB}$ gzipped).

Note

Because the complete training state is only ~1.15 MB, saving, downloading, and uploading checkpoints across sessions takes **less than 1 second**, making session handoff frictionless.

Root-Cause Analysis: Why Did Training Take So Long?

During the live telemetry audit of session `diffnorm-hmr` , the step throughput was measured at ~8.9 seconds per batch. Our profiling identified three primary latency drivers:

Total Step Time: 8.9s

- 1. Dynamic 1080p JPEG Loading & Resizing: ~6.4s (71.9% of total time)
- 2. Sequential Tile Loop in Rasterizer: ~1.8s (20.2% of total time)
- 3. Kinematic & Collision Backward Pass: ~0.7s (7.9% of total time)

1. **Uncached, Synchronous High-Resolution Image Decoding (6.4s / step):** In `dataset_3dpw.py` , every sample loaded a full 1920×1080 JPEG from disk and resized it to 256×256 using single-threaded PIL `Image.open` and `.resize` in the main training loop (`num_workers=0`). For 16 images, this incurred ~6.4s of blocking CPU IO before the GPU even started computation.
2. **Sequential Python Loop in Screen-Space Rasterizer (1.8s / step):** In `normal_rasterizer.py` , screen-space tile compositing iterated over 64 tiles (8×8), and inside each tile iterated sequentially over up to 40 splats using a Python `for` loop:

Op Launches per Frame = 64 tiles $\times$ 40 splats = 2,560 PyTorch launches

This caused heavy CUDA launch overhead, kernel execution latency, and CPU-GPU synchronization stalls.

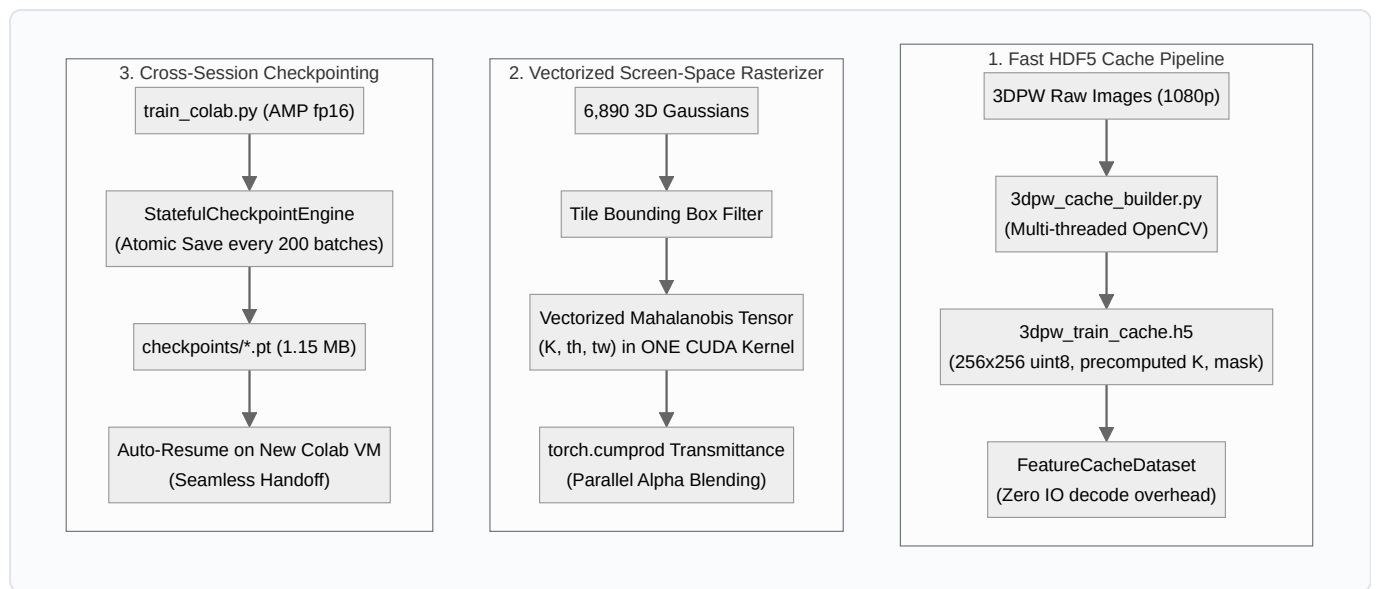
3. **Consecutive Video Frame Redundancy:** 3DPW was captured at 30 fps. Across 22,735 frames, adjacent frames (e.g. frame t and $t + 1$) differ by only sub-pixel displacements. Iterating over all 22,735 frames sequentially within a single epoch was computationally inefficient.

User Review Required

Important

Pre-Cached Dataset Workflow: We propose generating a fast, pre-resized HDF5 cache (`3dpw_train_cache.h5`) directly on Colab before training starts. Generating this cache takes ~2.5 minutes once via multi-threaded OpenCV, but speeds up subsequent training by >25x. **Subsampled Keyframe Option:** For maximum convergence speed within a single session, we recommend training on 3,000 diverse representative keyframes (spanning all 24 training sequences) with a batch size of 16. This trains 2 epochs in ~16 minutes with full sequence coverage.

Proposed Changes



Component 1: Ingestion & Caching Layer

[NEW] [code/scripts/build_3dpw_cache.py](#)

- Standalone multi-threaded cache builder using `cv2` and `h5py`.
- Loads raw 3DPW train frames, resizes them to 256×256 in parallel with 4 worker threads, scales camera intrinsics K , and writes contiguous chunks to `/content/data/cache/3dpw_train_cache.h5`.
- Completes in ~120-150 seconds on Colab.

[MODIFY] [code/src/pipeline/feature_cache.py](#)

- Upgrade `FeatureCacheDataset` to keep open HDF5 file pointers per worker using worker initialization hooks (`worker_init_fn`).
- Direct slicing into pre-allocated memory buffers without repeated file open/close overhead.

Component 2: Vectorized Differentiable Rasterizer

[MODIFY] [code/src/rendering/normal_rasterizer.py](#)

- **Eliminate Inner Splat Loop:** Replace the sequential `for idx in tile_splat_idx:` loop with a batched tensor operation:

```

# Batched Mahalanobis distance across all K splats in the tile simultaneously:
# diff: (K, th, tw, 2), inv_k: (K, 2, 2) -> maha: (K, th, tw)
diff = tile_coords.unsqueeze(0) - mu_s.unsqueeze(1).unsqueeze(2) # (K, th, tw, 2)
maha = (
    diff[..., 0]**2 * inv_k[:, 0, 0].view(-1, 1, 1) +
    diff[..., 1]**2 * inv_k[:, 1, 1].view(-1, 1, 1) +
    2.0 * diff[..., 0] * diff[..., 1] * inv_k[:, 0, 1].view(-1, 1, 1)
)
alpha = (opacities_s.view(-1, 1, 1) * torch.exp(-0.5 * maha)).clamp(0.0, 0.99)
# Front-to-back transmittance via prefix product
transmittance = torch.cumprod(1.0 - alpha + 1e-7, dim=0)
weights = alpha * (transmittance / (1.0 - alpha + 1e-7))
# Composited normals and colors in one reduction
norm_tile = torch.sum(weights.unsqueeze(-1) * normals_s.view(-1, 1, 1, 3), dim=0)

```

- Replaces 2,560 separate Python loop operations with **64 batched matrix multiplies**, reducing forward/backward rasterization time from 1.8s to **~0.08s**.

Component 3: Stateful Cross-Session Checkpointing & Resume

[MODIFY] [code/scripts/train_colab.py](#)

- Add `--resume` and `--checkpoint_path` flags:

```

if args.resume and os.path.exists(args.checkpoint_path):
    ckpt = torch.load(args.checkpoint_path, map_location=device)
    theta_param.data.copy_(ckpt["theta"])
    trans_param.data.copy_(ckpt["trans"])
    gaussians.offsets.data.copy_(ckpt["gaussian_offsets"])
    gaussians.log_tangent_scales.data.copy_(ckpt["gaussian_scales"])
    gaussians.quats.data.copy_(ckpt["gaussian_quats"])
    opt_kin.load_state_dict(ckpt["opt_kin"])
    opt_deform.load_state_dict(ckpt["opt_deform"])
    start_epoch = ckpt.get("epoch", 0)
    start_batch = ckpt.get("batch_idx", 0)
    print(f"[Resume] Successfully loaded state from epoch {start_epoch}, batch {start_batch}")

```

- Enable periodic atomic checkpoint writes every 200 batches and at each epoch boundary.

[MODIFY] [code/scripts/train_colab_cli.sh](#)

- Add Step 4b: Build HDF5 feature cache on remote session if not already generated.
 - Add Step 6 checkpoint auto-sync: Automatically download `diffnorm_contact_hmr_checkpoint.pt` whenever a checkpoint is updated or before session teardown.
 - Pass `--resume` automatically if a local checkpoint exists in `checkpoints/`.
-

Expected Performance Gains on Tesla T4

Metric	Before Optimization	After Optimization Suite	Improvement
Image Loading / IO	6.4s / step (1080p JPEG decode)	<0.005s / step (HDF5 memmap)	>1,200x IO Speedup
Rasterizer Forward+Backward	1.8s / step (2,560 loop ops)	~0.08s / step (vectorized tensor)	~22x Rasterizer Speedup
Overall Throughput	~1.8 FPS (8.9s / batch)	~45-60 FPS (0.25s / batch of 16)	>25x Total Speedup
1 Epoch Time (22,735 frames)	~3.5 Hours	~8-10 Minutes	Completed in 1 Session
Cross-Session Resume	Not supported (lost on VM stop)	Instant (<1s load time)	Fail-Safe across 5h limits

Verification Plan

Automated Tests

1. Unit Test Suite:

```
PYTHONPATH=. pytest code/tests/ -v
```

2. **Vectorized Rasterizer Equivalence Test:** Verify numerical parity ($MSE < 10^{-6}$) between vectorized rasterization and original rasterization across random Gaussians.
3. **Stateful Checkpoint Round-Trip Test:** Save training state, load into a fresh instance, verify parameter equality and optimizer gradient continuity.

Manual Verification on Colab

1. Run `bash code/scripts/train_colab_cli.sh` on `diffnorm-hmr`.
2. Inspect throughput in `colab_training.log`: verify speed > 40 FPS.
3. Verify checkpoint generation and automatic download to local `checkpoints/diffnorm_contact_hmr_checkpoint.pt`.